Ders 1 Genişletilmiş Notu: Derse Giriş: Algoritma Kavramı, Büyüme Mertebeleri ve En Kötü, En İyi, Ortalama Durum

Ders 1 Genişletilmiş Notu: Derse Giriş: Algoritma Kavramı, Büyüme Mertebeleri ve En Kötü, En İyi, Ortalama Durum

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Kaynakları
- 2. Dönem Boyunca İşlenecek Konular
- 3. Algoritma Kavramı ve İyi Bir Algoritmanın Özellikleri
- 4. Temel Veri Yapıları
- 5. Çalışma Zamanını Etkileyen Etkenler
- 6. Girdi Boyutu ve Çalışma Zamanının Ölçülmesi
- 7. Büyüme Mertebeleri
- 8. En Kötü, En İyi ve Ortalama Durum
- 9. Ders Sonundaki Soru-Cevap
- Örnek Sorular

1. Dersin Kaynakları

Bu ilk ders kısa bir giriş dersidir. Amaç dersin içeriğinden hızlıca geçmek ve temel kavramları hatırlatmaktır. Kapak slaytında dersin adı “BLM5106 - Advanced Algorithm Analysis and Design”, bölüm olarak da Yıldız Teknik Üniversitesi Bilgisayar Mühendisliği yazıyor (ekrandan, 00:07:00).

Slayttaki kaynaklar şunlardır (ekrandan, 00:07:58):

- Anany Levitin, “Introduction to the Design & Analysis of Algorithms”, 2011.
- Cormen, Leiserson, Rivest, Stein, “Introduction to Algorithms, 3E”, MIT Press, 2009.
- R. Sedgewick ve K. Wayne, “Algorithms, Fourth Edition”, 2013 (http://algs4.cs.princeton.edu).
- Steven Skiena, “The Algorithm Design Manual”, 2010.

Hoca, internette bolca kaynak olduğunu ve lisansüstü öğrencilerin güvenilir kaynağı bulmayı bildiğini söyledi. Yine de özellikle ilk kitabın (Levitin) çok iyi düzenlendiğini ve derste ondan sık faydalanacağını belirtti.

2. Dönem Boyunca İşlenecek Konular

Öğrenciler farklı okullardan ve farklı disiplinlerden geldiği için hoca herkesi ortak bir temele oturtmak istiyor. Bu yüzden ders, bilinen temel kavramların hatırlatılmasıyla başlayacak. Temel konuların geçilme hızı öğrencilerden gelen geri bildirime göre ayarlanacak; hoca dersin etkileşimli olmasını rica etti.

İçerik slaytındaki sıra şöyledir (ekrandan, 00:08:50 ve 00:12:30):

- **1. Algoritmaların temelleri** (Fundamentals for Algorithms).

- **2. Algoritma verimliliği analizinin temelleri, asimptotik analiz:** algoritma analizi, karmaşıklıklar, Big O, Big Theta, Big Omega, büyüme mertebeleri.
- **3. Özyinelemeli olmayan ve özyinelemeli algoritmaların analizi:** çalışma zamanı, recurrence relation, backward substitution. Hoca, recurrence relation'ın nasıl çıkarıldığını ve bir başlangıç değeri (initial value) için backward substitution ile kapalı formülün nasıl bulunduğunu hatırlatacağını söyledi.
- **4. Böl ve fethet algoritmalarının analizi:** brute force, exhaustive search, decrease and conquer. Önce brute force ve exhaustive search anlatılacak, sonra bunlar böl ve fethet algoritmalarıyla karşılaştırılarak analizleri yorumlanacak. Hoca, böl ve fethete benzeyen decrease and conquer ve transform and conquer gibi yaklaşımlardan da bahsedeceğini söyledi.
- **5. Hashing algoritmaları:** hoca open hashing (bağlı listelerden faydalanma) ve closed hashing'e değinileceğini söyledi.
- **6. Dinamik programlama:** rod cutting, matrix-chain multiplication, dinamik programlamanın öğeleri, optimal binary search trees.
- **7. Açgözlü (greedy) algoritmalar:** activity-selection problem, açgözlü stratejinin öğeleri.
- **8. Ara sınav.**
- **9. Amortize analiz:** aggregate analysis, accounting method, potential method, dynamic tables.
- **10. İleri veri yapıları:** B-trees, Fibonacci heaps, AVL trees.
- **11. Temel graf algoritmaları:** graf gösterimleri, breadth-first search, depth-first search, topological sort, strongly connected components.
- **12. Sunumlar.**
- **13. String matching:** naive string-matching algoritması, Rabin-Karp, sonlu otomatla string matching, Knuth-Morris-Pratt.
- **14. Yaklaşık (approximation) algoritmalar:** vertex-cover, traveling-salesman, set-covering, randomization ve linear programming, subset-sum.

Hoca bu başlıkların bazılarına kısa açıklamalar ekledi:

- Böl ve fethet ile hashing konuları lisanstaki veri yapıları ve algoritma analizi derslerinden tanıdık olmalıdır. Bütün algoritmalar her seferinde karmaşıklık açısından da incelenecek: önce algoritma anlatılacak, sonra karmaşıklığı konuşulacak.
- **Dinamik programlama:** "programming" kelimesi burada aslında planlamayı temsil eder. Dinamik programlamada alt problemler (subproblem) tanımlanır ve bunlar çözülür. Kavram, rahat anlaşılmasını sağlayacak örnekler üzerinden anlatılacak. Dinamik programlama optimizasyon problemleri için kullanılan bir yöntemdir; hangi tür problemlerin dinamik programlamayla çözülmesi gerektiği de araştırılacak.
- **Açgözlü algoritmalar:** o anki en iyi çözüm üzerinden ilerler. Nasıl çalıştıkları activity-selection problemi üzerinden anlatılacak. Ardından açgözlü stratejinin ne zaman kullanılması gerektiği, yani bir problemin bu yolla çözülebilmesi için hangi özellikleri taşıması gerektiği konuşulacak.
- **Amortize analiz:** zaman karmaşıklığına dair bir yöntemdir. En kötü durum analizi karamsardır (pessimistic); amortize analiz bunu iyileştirip gerçeğe daha yakın sonuç elde etmek için önerilmiştir. Uygulama yolları olarak aggregate analysis ve accounting method anıldı. Dinamik tablolar amortize analizin kullanılmasına çok uygun bir problemdir; üzerlerindeki işlemlerle uygulama örnekleri verilecek.
- **İleri veri yapıları:** öğrencilerin daha önce karşılaşmamış olabileceği yapılar anlatılacak: dengeli arama ağaçları, AVL ağacı (kendini dengeleyen ikili arama ağacı), Fibonacci heap. Bu konu bir iki haftaya yayılabilir.
- **Graf algoritmaları:** graf üzerinde arama için breadth-first search, depth-first search ve topological sort hatırlatılacak.
- **Sunumlar:** çok farklı algoritmalar olduğu ve hepsine değinmek mümkün olmadığı için, hocanın atadığı ya da öğrencilerin seçtiği algoritmalar üzerine sunumlar yapılması planlanıyor.
- **String matching:** neden önemli olduğuna iki örnek verildi: bir DNA dizisinde belirli bir örüntü (pattern) aramak ve internette arama yapıp ilgili web sayfalarını bulmak. Brute force string matching ve Rabin-Karp gibi algoritmalar örneklerle anlatılacak.
- **Yaklaşık algoritmalar:** NP-complete problemler için geliştirilmiştir. Amaçları optimale yakın (near-optimal) çözümler bulmaktır. Birkaç farklı problem üzerinden konuşulacak.

3. Algoritma Kavramı ve İyi Bir Algoritmanın Özellikleri

3.1 Algoritmanın tanımı

Hoca bu kısmı bir "Hello World" tadında başlangıç olarak sundu. Slayttaki tanım şöyledir (ekrandan, 00:15:30):

> An algorithm is a sequence of unambiguous instructions for solving a problem, for obtaining a required output for any legitimate input in a finite amount of time.

Yani algoritma, bir problemi çözmek için, sınırları belli geçerli (legitimate) her girdiden istenen çıktıyı sonlu bir sürede elde eden ve belirsizlik içermeyen komutlar dizisidir. Slayttaki şemada problemden algoritmaya, algoritmadan "bilgisayar" kutusuna ok iner; girdi bu kutuya girer, kutudan çıktı çıkar.

3.2 İyi bir algoritmadan beklenenler

Slaytın başlığı "Analyzing an Algorithm"dır (ekrandan, 00:16:10). İlk soru "algoritma ne kadar iyi?" sorusudur:

- **Doğruluk (correctness):** mümkün olduğunca doğru sonuç vermeli.
- **Verimlilik (efficiency):** zaman ve bellek (space) açısından. Slaytta kırmızıyla şu not var: bir algoritma polinom çalışma zamanına sahipse verimli sayılır. Zaman ve bellek üzerinde analiz bu dersin konusudur.
- **Sadelik (simplicity):** problemi basitçe çözebilmeli.
- **Genellik (generality):** problemi genelleştirebilmeli.

İkinci soru "daha iyi bir algoritma var mı?" sorusudur; bu da alt sınırlar (lower bounds) ve optimallik (optimality) kavramlarıyla ilgilidir. Hoca, daha düşük karmaşıklıklı, daha optimum algoritmaların daha iyi sayıldığını ve bu özellikleri nesnel olarak değerlendirmenin de dersin konusu olduğunu söyledi.

4. Temel Veri Yapıları

Derste çözülecek örnekler temel veri yapıları üzerine kuruludur; bunlardan daha karmaşık yapılar da gelecektir. Bu yüzden bu veri yapılarını hatırlamak gerekir. Hoca, öğrencilerin bunları lisans eğitiminde mutlaka gerçeklediğini, gerekirse bu haftayı bunları hatırlayarak geçirebileceklerini söyledi. Slayttaki liste şöyledir (ekrandan, 00:16:40):

- **Doğrusal veri yapıları:** diziler, bağlı listeler, yığın (stack), kuyruk (queue).
- **Graflar:** düğümler, kenarlar, komşuluk matrisi (adjacency matrix) ve komşuluk listeleri (adjacency lists).
- **Ağaçlar:** köklü ağaçlar (rooted trees), sıralı ağaçlar (ordered trees).
- **Kümeler ve sözlükler:** evrensel küme (universal set), bit vektörü, liste yapısının kullanılması.

5. Çalışma Zamanını Etkileyen Etkenler

Bir algoritmanın çalışma hızı iki grup etkene bağlıdır (ekrandan, 00:17:30):

- **Sistemden bağımsız etkenler:** algoritmanın kendisi. Bu dersin konusu budur.
- **Sisteme bağlı etkenler:**
  - Donanım: CPU, bellek, önbellek (cache).
  - Yazılım: derleyici (compiler), yorumlayıcı (interpreter) vb.
  - Sistem: işletim sistemi, ağ vb.

Algoritmanın pratikteki etkinliği bunların hepsinin bir bütünü olarak şekillenir. Ders, sistemden bağımsız olan kısmı, yani algoritmayı inceler.

6. Girdi Boyutu ve Çalışma Zamanının Ölçülmesi

6.1 Girdi boyutu

Bir algoritmanın çalışma zamanı ölçülmek istendiğinde en önemli parametre girdi boyutudur (input size); çalışma zamanını doğrudan etkiler. Slayttaki iki örnek (ekrandan, 00:18:40):

- İki matrisin çarpımında girdi boyutu, çarpılan matrislerdeki toplam eleman sayısı N'dir.
- Bir polinomun değerlendirilmesinde, yani verilen bir x için polinomun değerinin hesaplanmasında, girdi boyutu polinomun derecesidir.

6.2 Çalışma zamanının birimi

Çalışma zamanı n'e bağlı bir fonksiyon olarak $T(n)$ ile gösterilir ve yaklaşık olarak şöyle bulunur (ekrandan, 00:18:40):

$$T(n) \approx c_{op}\, C(n)$$

- c_{op}: algoritmadaki temel işlemin (basic operation) bir kez çalıştırılması için geçen süredir.
- $C(n)$: bu temel işlemin algoritmada n'e bağlı olarak kaç kez yapılacağını veren fonksiyondur.

Bu ikisi çarpılınca çalışma zamanı bulunur. Sonuç kesin değil, yaklaşıktır. $C(n)$'in nasıl çıkarıldığı sonraki derslerde anlatılacak.

6.3 Girdi iki katına çıkınca çalışma süresi

Slayttaki soru (ekrandan, 00:18:40): $C(n) = \frac{1}{2}n(n-1)$ olduğunu varsayalım. Girdi boyutu iki katına çıkarılırsa algoritma ne kadar uzun çalışır?

Adım 1, $C(n)$'i açma ve sadeleştirme:

$$C(n) = \frac{1}{2}n(n-1) = \frac{1}{2}n^2 - \frac{1}{2}n \approx \frac{1}{2}n^2$$

İfadede n'e bağlı iki terim vardır ve birinin üssü daha yüksektir. Yüksek üslü terimi baz almak bir şey kaybettirmez, çünkü ölçülmek istenen n sonsuza giderken algoritmanın nasıl etkilendiğidir.

Adım 2, oranı hesaplama:

$$\frac{T(2n)}{T(n)} \approx \frac{c_{op}\, C(2n)}{c_{op}\, C(n)} \approx \frac{\frac{1}{2}(2n)^2}{\frac{1}{2}n^2} = 4$$

c_{op}'ler, $\frac{1}{2}$'ler ve n^2'ler birbirini götürür; geriye 4 kalır. Girdi iki katına çıkınca çalışma süresi yaklaşık 4 katına çıkar.

Bu örnek neyi gösteriyor: Analizde sabit çarpanlarla ilgilenilmez. Temel işlemin çalışma süresiyle (c_{op}) de ilgilenilmez. İlgilenilen şey, girdi boyutu büyüdüğünde yapılacak işin kaç kat arttığıdır. Bu orana büyüme mertebesi (order of growth) denir ve dersteki analizler bunun üzerinden yapılacaktır.

7. Büyüme Mertebeleri

Slayttaki tablo, temel algoritma sınıflarının büyüme mertebesini, tipik kod yapısını ve girdi iki katına çıkınca çalışma süresinin oranını ($T(2N)/T(N)$) gösterir (ekrandan, 00:21:40):

Mertebe	Ad	Tipik kod yapısı	Açıklama	Örnek	$T(2N)/T(N)$
1	sabit (constant)	`a = b + c;`	ifade	iki sayıyı toplama	1
$\log N$	logaritmik	`while (N > 1) { N = N / 2; ... }`	yarıya bölme	binary search	~1
N	doğrusal (linear)	`for (int i = 0; i < N; i++) { ... }`	döngü	en büyüğü bulma	2
$N \log N$	linearithmic	[mergesort dersine bakın]	böl ve fethet	mergesort	~2
N^2	karesel (quadratic)	iç içe iki `for`	çift döngü	tüm çiftleri kontrol etme	4
N^3	kübik (cubic)	iç içe üç `for`	üçlü döngü	tüm üçlüleri kontrol etme	8
2^N	üstel (exponential)	[combinatorial search dersine bakın]	exhaustive search	tüm alt kümeleri kontrol etme	(boş)

Hocanın tabloya eklediği açıklamalar:

- `a = b + c` gibi tek bir ifadenin büyüme mertebesi sabittir.
- Binary search her seferinde ikiye bölerek çalışır; bu yüzden logaritmik bir fonksiyon ortaya çıkar.
- Basit bir dizi üzerinde sıralı arama (sequential search) doğrusal artar: girdi iki katına çıkınca çalışma süresi de iki katına çıkar.
- Böl ve fethet algoritmalarından mergesort'un büyüme mertebesi $N \log N$'dir.
- Bir dizide birbiriyle aynı olan çiftleri bulmak için iç içe iki `for` kullanılırsa karesel bir fonksiyon oluşur; girdi iki katına çıkınca süre dört katına çıkar. Kübik fonksiyonda sekiz katına çıkar.

  Hocanın vurgusu: Tercih edilen logaritmik olmasıdır; olmazsa doğrusal. Karesel de kabul edilebilir, ama n arttıkça çalışma süresi bir süre sonra çok hızlı büyümeye başlar; iki katı yerine on katı, yüz katı girdiyi düşünmek gerekir. Üstel olduktan sonra algoritma ya mutlaka optimize edilmeli ya da yalnız çok küçük girdi boyutlarında kullanışlı sayılabilir.

  Ek bilgi (derste anlatılmadı): Tablodaki "~1" ve "~2" değerleri yaklaşıktır. Örneğin $\frac{\log(2N)}{\log N} = 1 + \frac{\log 2}{\log N}$ olduğundan oran N büyüdükçe 1'e yaklaşır; benzer biçimde $N \log N$ için oran $2\left(1 + \frac{\log 2}{\log N}\right)$ olup 2'ye yaklaşır.

8. En Kötü, En İyi ve Ortalama Durum

8.1 Sıralı arama algoritması

Kavramlar sıralı arama üzerinden hatırlatıldı. Elimizde bir dizi vardır ve verilen bir anahtar (key) aranır. Elemanlara tek tek bakılır: bir `while` döngüsü dizinin sonuna gelene ya da anahtar bulunana kadar döner. Döngüden çıkınca neden çıkıldığı kontrol edilir; bulunduysa indis, bulunamadıysa -1 döndürülür. Slayttaki algoritma şöyledir (ekrandan, 00:23:50):

```
ALGORITHM SequentialSearch(A[0..n-1], K)
// Searches for a given value in a given array by sequential search
// Input: An array A[0..n-1] and a search key K
// Output: The index of the first element in A that matches K
//         or -1 if there are no matching elements
i <- 0
```

```
while i < n and A[i] != K do
    i <- i + 1
if i < n return i
else return -1
```

8.2 En kötü ve en iyi durum

- **En kötü durum:** $C_{worst}(n) = n$. Döngü n kez döner, çünkü bütün dizi taranır ve anahtar ya bulunamaz ya da en sonda bulunur.
- **En iyi durum:** $C_{best}(n) = 1$. Anahtar ilk elemandır.

Pratikte daha çok en kötü durum dikkate alınır. Ama bu, en iyi durumun kullanışsız olduğu anlamına gelmez. Örneğin insertion sort ile neredeyse sıralı bir dizi sıralanıyorsa, gerçekleşen senaryo en iyi duruma yakındır. Böyle bir problemde algoritmayı seçerken en iyi durumu değerlendirmek daha faydalı olur.

> **Ek bilgi (derste anlatılmadı):** Insertion sort'un en iyi durumu zaten sıralı dizidir; her eleman yalnız bir önceki elemanla karşılaştırılır ve toplam $n - 1$ karşılaştırma yapılır. En kötü durumu (ters sıralı dizi) ise karesel sayıda karşılaştırma gerektirir.

8.3 Ortalama durum

Ortalama durum, en kötü durumun fazla karamsar olduğu düşüncesiyle daha gerçekçi bir sonuç bulmayı hedefler. Hesaplamak için bazı varsayımlar yapmak gerekir. Bu örnekte iki varsayım vardır (ekrandan, 00:26:40):

- (a) Başarılı bir aramanın (aranan elemanın dizide olmasının) olasılığı p'dir, $0 \leq p \leq 1$.
- (b) İlk eşleşmenin i. konumda olma olasılığı her i için aynıdır (uniform).

Hesap, terim terim:

- Eleman dizideyse n konumdan herhangi birinde eşit olasılıkla bulunur; her konum için bu olasılık $\frac{p}{n}$'dir.
- Birinci konumda bulunma olasılığı $\frac{p}{n}$, bulana kadar yapılan işlem sayısı 1'dir: terim $1 \cdot \frac{p}{n}$.
- İkinci konumda bulmak için döngü iki kez döner: terim $2 \cdot \frac{p}{n}$.
- Üçüncü konum için $3 \cdot \frac{p}{n}$; genel olarak i. konum için $i \cdot \frac{p}{n}$; sonuncusu $n \cdot \frac{p}{n}$.
- Bulunamama olasılığı $1 - p$'dir. Bulamamak için döngü n kez döner: terim $n(1 - p)$.

Bunlar toplanınca slayttaki formül elde edilir:

$$C_{avg}(n) = \left[1 \cdot \frac{p}{n} + 2 \cdot \frac{p}{n} + \cdots + i \cdot \frac{p}{n} + \cdots + n \cdot \frac{p}{n}\right] + n(1 - p)$$

Hoca bu formülün birkaç adımda daha anlaşılır bir hâle sadeleştirilebileceğini söyledi. Sadeleştirme (ara adım eklendi):

$$C_{avg}(n) = \frac{p}{n}\left(1 + 2 + \cdots + n\right) + n(1 - p) = \frac{p}{n} \cdot \frac{n(n+1)}{2} + n(1 - p) = \frac{p(n+1)}{2} + n(1 - p)$$

Bu hesap en kötü ve en iyi duruma göre bir tık daha karmaşıktır. Ama hocaya göre burada asıl ilgilenilen şey formülü hesaplamak değil, p'nin durumudur.

p'nin uç değerleri:

- $p = 1$ ise (anahtar mutlaka dizide): $C_{avg}(n) = \frac{n+1}{2}$. Yani anahtar, ortalama olarak dizinin ortalarına gelindiğinde bulunur.
- $p = 0$ ise (anahtar dizide yok): köşeli parantez içindeki kısım sıfır olur ve $C_{avg}(n) = n$'dir.

> **Hocanın vurgusu:** Ortalama durumdan fayda sağlamak için p'nin durumunu iyi değerlendirmek gerekir. p seçilirken dizinin dağılımı göz önünde bulundurulmalıdır.

Örneğin Gauss dağılımına uyan bir dizide her eleman için ayrı ayrı p belirlemek yerine dizi küçük parçalara ayrılıp her parça (slot) için bir p belirlenebilir. Bu seçimler, analizin ne kadar başarılı olduğunu doğrudan değiştirir.

9. Ders Sonundaki Soru-Cevap

Soru: Lisansta veri yapılarını aldım ama algoritma analizi dersini almadım. Bu ders algoritma analizine çok bağlı mı?

Cevap: Algoritma analizi dersini almış olmak avantaj sağlar. Ama ders içeriği, lisansta bu dersi veren hocalarla iletişim hâlinde hazırlandı: ilk birkaç hafta onlara paralel, farklı örnekler çözülecek. Böylece dersi almayanlar öğrenmiş, alanlar hatırlamış olacak. En basit özyinelemeli olmayan ve özyinelemeli algoritmaların nasıl analiz edildiğinden başlanıp adım adım ilerlenecek, ama lisanstaki gibi bütün döneme yayılmayacak. İlk haftalar ve kaynak kitaplar iyi takip edilirse sorun çıkmaz.

Soru: Dersin slaytları bir yerde var mı?

Cevap: Ders her zaman slaytla işlenmeyebilir. Özellikle formüllerin olduğu kısımlar tablet üzerinde yazılarak anlatılabilir; bu yüzden not almak gerekecek. Bu giriş slaytı paylaşılacak.

Örnek Sorular

Soru 1 (ekrandan, 00:18:40): $C(n) = \frac{1}{2}n(n-1)$ olduğunu varsayalım. Girdi boyutu iki katına çıkarılırsa algoritma ne kadar uzun çalışır?

> **Çözüm (derste verildi):** $C(n) = \frac{1}{2}n^2 - \frac{1}{2}n \approx \frac{1}{2}n^2$. $\frac{T(2n)}{T(n)} \approx \frac{c_{op}C(2n)}{c_{op}C(n)} \approx \frac{\frac{1}{2}(2n)^2}{\frac{1}{2}n^2} = 4$.
> Algoritma yaklaşık 4 kat uzun çalışır.

Soru 2 (ekrandan, 00:21:40): Girdi boyutu iki katına çıkınca sabit, logaritmik, doğrusal, $N \log N$, karesel ve kübik büyüme mertebesine sahip algoritmaların çalışma süresi kaç katına çıkar?

> **Çözüm (derste verildi):** Slayttaki $T(2N)/T(N)$ sütununa göre sırasıyla 1, ~1, 2, ~2, 4 ve 8.

Soru 3 (derste sözlü, 00:28:01): Sıralı aramanın ortalama durum formülünde $p = 1$ ise, yani aranan eleman mutlaka dizideyse, ortalama durum neye gider? $p = 0$ ise neye gider?

> **Çözüm (derste verildi):** $C_{avg}(n) = \frac{p(n+1)}{2} + n(1-p)$. $p = 1$ için $\frac{n+1}{2}$: eleman ortalama olarak dizinin ortasında bulunur. $p = 0$ için köşeli parantezli kısım sıfır olur ve sonuç n'dir.

Dersin Toparlaması

Bu giriş dersinde önce dönem boyunca işlenecek konular gözden geçirildi: asimptotik analiz, özyinelemeli ve özyinelemeli olmayan algoritmaların analizi, böl ve fethet, hashing, dinamik programlama, açgözlü algoritmalar, amortize analiz, ileri veri yapıları, graf algoritmaları, string matching ve yaklaşık algoritmalar. Ardından algoritmanın tanımı, iyi bir algoritmanın özellikleri ve çalışma zamanını etkileyen etkenler hatırlatıldı.

Dersin teknik çekirdeği iki fikirdi. Birincisi, çalışma zamanı $T(n) \approx c_{op}C(n)$ ile modellenir ve analizde sabitler değil, girdi büyüdüğünde işin kaç kat arttığı, yani büyüme mertebesi önemlidir. İkincisi, aynı algoritmanın en kötü, en iyi ve ortalama durumları farklıdır; sıralı aramada bunlar n, 1 ve $\frac{p(n+1)}{2} + n(1-p)$'dir ve ortalama durumda asıl iş p'yi verinin dağılımına göre doğru seçmektir. Hoca, sonraki derste temel veri yapılarının ve yüksek dereceli terimin neden baz alındığının ayrıca anlatılacağını belirtti.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.